



SWAYAM NPTEL COURSE ON STATISTICAL FOUNDATION FOR BIG DATA ANALYSIS

BY

PROF. ARINDAM BANERJEE

Department of Mathematics
INDIAN INSTITUTE OF TECHNOLOGY KHARAGPUR

**Module 08: Multivariate Analysis And
Dimensionality**

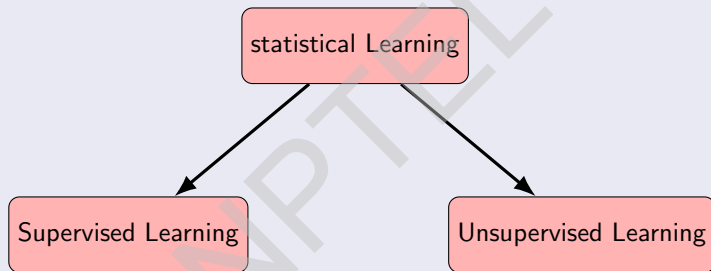
**Lecture 01: Supervised Learning vs Unsupervised
Learning**



What we have done so far

- Statistical framework from population data via random variable.
- Statistical inference :
 - Estimation.
 - Hypothesis testing.
- Framework of statistical learning theory.
- Multiple linear regression.
- Multivariate normal distribution.

Types of Statistical Learning



Supervised Learning

- Supervised learning refers to learning a mapping from inputs to outputs using labeled data.
- The training dataset consists of pairs (x_i, y_i) , where x_i is the input and y_i is the target output.
- The objective is to learn a function $f(x)$ such that $f(x_i)$ approximates y_i for training samples and generalizes well to unseen data.
- Supervised learning tasks are mainly of two types:
 - *Linear Regression*: Predicting continuous outputs.
 - *Logistic Regression*: Predicting categorical outputs.
- The learning process typically involves:
 - Selecting a model family (e.g., linear models, neural networks).
 - Defining a loss function to measure prediction error.
 - Optimizing model parameters using training data.

Supervised Learning (continued)

- Common algorithms include:
 - Linear regression
 - Logistic regression
 - k-nearest neighbors
 - Support vector machines
 - Decision trees and random forests
 - Neural networks
- Model performance is evaluated using metrics such as:
 - Mean squared error (regression)
 - Accuracy, precision, recall, F1-score (classification)
- Cross-validation is often used to estimate generalization performance and prevent overfitting.

Unsupervised Learning

- Unsupervised learning deals with data that does not have labeled outputs.
- The goal is to discover hidden structure or patterns within the data.
- The dataset consists only of inputs $x_1, x_2, \dots, x_n$, with no corresponding target values.
- Common unsupervised learning tasks include:
 - *Clustering*: Grouping similar data points together.
 - *Dimensionality reduction*: Finding a lower-dimensional representation of data.
- Examples of clustering algorithms include:
 - K-means clustering
 - Hierarchical clustering

Unsupervised Learning (continued)

- Common dimensionality reduction techniques include:
 - Principal component analysis (PCA)
 - Matrix factorization.
- Evaluation is more challenging than supervised learning because no labels are available.
- Unsupervised learning can be used for exploratory data analysis, anomaly detection, preprocessing, and visualization.
- It often serves as a preprocessing step for supervised learning by extracting features or reducing noise.

Differences between Supervised and Unsupervised Learning

<i>Supervised Learning</i>	<i>Unsupervised Learning</i>
Uses labeled data with input-output pairs (x_i, y_i)	Uses only input data without labels
Learns a mapping from inputs to outputs	Learns hidden structure or patterns in the data
Main tasks: regression and classification	Main tasks: clustering, dimensionality reduction, density estimation
Model performance can be directly evaluated using metrics like accuracy or MSE	Evaluation is more difficult because no label is available.
Examples: linear regression, logistic regression, SVM, neural networks	Examples: K -means, hierarchical clustering, PCA, Gaussian mixtures
Goal: predict labels for new data	Goal: understand data structure or group similar instances

Advantages of Supervised Learning

- 1 Accurate and Reliable Predictions. (Example: A handwriting recognition system correctly identifies handwritten numbers on checks.)
- 2 Easy to Understand and Implement. (Example: Predicting student grades based on attendance and previous performance.)
- 3 Useful for Predictive Analytics. (Example: Predicting future sales based on past purchase data.)
- 4 Automates Repetitive Tasks. (Example: An AI-powered chatbot providing customer support by answering FAQs.)

Disadvantages of Supervised Learning

- 1 Requires Large Amounts of Labeled Data. (Example: A model detecting fake news needs thousands of labeled articles for training.)
- 2 Limited to Patterns in Training Data. (Example: A language model trained on English texts won't perform well on Spanish text.)
- 3 Overfitting Risk. (Example: A facial recognition system trained only on daylight images may fail at night.)
- 4 Not Suitable for Dynamic and Rapidly Changing Environments. (Example: Stock market prediction models need frequent updates due to market changes.)
- 5 Struggles with Unstructured Data. (Example: Translating handwriting into digital text is harder than classifying numbers.)
- 6 Human Bias in Training Data. (Example: A hiring AI trained on past male-dominated recruitment data may unfairly favor male candidates.)

Advantages of Unsupervised Learning

- 1 No Need for Labeled Data. (Example: A company can use unsupervised learning to group customers based on their buying patterns without knowing their preferences beforehand.)
- 2 Can Discover Hidden Patterns. (Example: Social media platforms use unsupervised learning to identify communities and trends.)
- 3 Works Well for Complex Data. (Example: Medical research uses unsupervised learning to group patients with similar symptoms for better treatment planning.)
- 4 Reduces Human Bias. (Example: In hiring processes, an unsupervised model can identify diverse talent groups without being influenced by human biases.)

Disadvantages of Unsupervised Learning

- 1 Results May Be Uninterpretable. (Example: A model may group customers into clusters, but understanding why these groups exist requires further analysis.)
- 2 Accuracy is Not Guaranteed. (Example: A clustering algorithm might group unrelated objects together, making the results less useful.)
- 3 Requires More Computational Power. (Example: Image recognition using unsupervised learning requires high-end hardware to process complex visual data.)
- 4 May Learn Incorrect or Irrelevant Patterns. (Example: If an unsupervised algorithm groups customers based on random behavior instead of meaningful characteristics, the insights will not be useful.)

References

- 1 A. Géron, Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow, 2nd ed. Sebastopol, CA, USA: O'Reilly Media, 2019.
- 2 A. C. Müller and S. Guido, Introduction to Machine Learning with Python: A Guide for Data Scientists. Sebastopol, CA, USA: O'Reilly Media, 2016.
- 3 A. Burkov, The Hundred-Page Machine Learning Book. 2019.

Thank you





SWAYAM NPTEL COURSE ON STATISTICAL FOUNDATION FOR BIG DATA ANALYSIS

BY

PROF. ARINDAM BANERJEE

Department of Mathematics
INDIAN INSTITUTE OF TECHNOLOGY KHARAGPUR

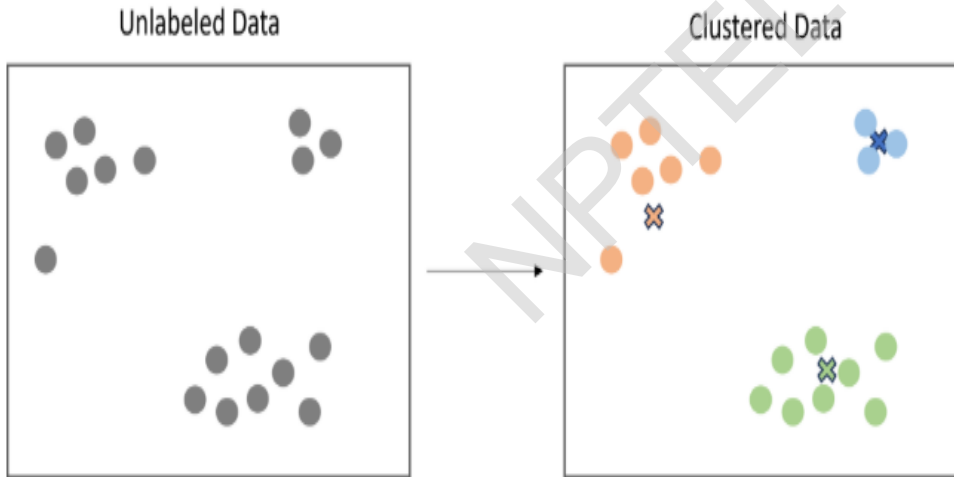
**Module 08: Multivariate Analysis And
Dimensionality**

Lecture 02: Hierarchical Clustering

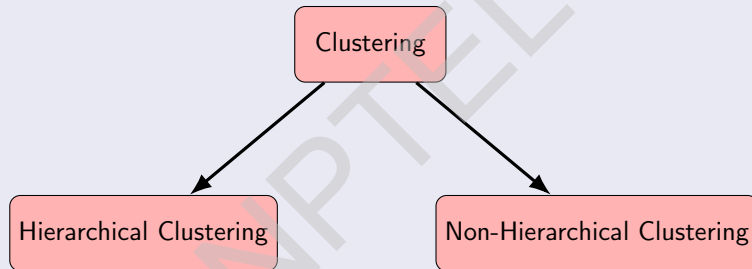


Clustering

Clustering is an unsupervised learning technique that is designed to group objects or observations together based on their similarities.



Types of Clustering



Hierarchical Clustering

Hierarchical clustering is a method of cluster analysis that seeks to build a hierarchy of clusters. Strategies for hierarchical clustering generally fall into two categories:

- Agglomerative:
 - Agglomerative clustering, often referred to as a "bottom-up" approach, begins with each data point as an individual cluster.
 - At each step, the algorithm merges the two most similar clusters based on a chosen distance metric (e.g., Euclidean distance) and linkage criterion (e.g., single-linkage, complete-linkage).
 - This process continues until all data points are combined into a single cluster or a stopping criterion is met.
 - Agglomerative methods are more commonly used due to their simplicity and computational efficiency for small to medium-sized datasets.
- Divisive:
 - Divisive clustering, known as a "top-down" approach, starts with all data points in a single cluster and recursively splits the cluster into smaller ones.
 - At each step, the algorithm selects a cluster and divides it into two or more subsets, often using a criterion such as maximizing the distance between resulting clusters.
 - Divisive methods are less common but can be useful when the goal is to identify large, distinct clusters first.

Differences with Non-Hierarchical Clustering

Feature	Hierarchical Clustering	Non-Hierarchical Clustering (e.g., k-means)
Structure	Produces a tree-like structure (dendrogram).	Produces a flat set of clusters.
Cluster Formation	Built step-by-step by merging or splitting clusters.	Clusters are formed by optimizing a criterion (e.g., minimizing within-cluster variance).
Number of Clusters	Not required in advance; can be chosen later from dendrogram.	Must specify number of clusters k in advance.

Agglomerative Hierarchical Clustering

- 1 Start with N clusters, each containing a single entry and $N \times N$ symmetric matrix of distances $D = d_{ik}$.
- 2 Search the distance matrix for the nearest pair of clusters. Let the distance between “most similar” clusters U and V be d_{UV} .
- 3 Merge clusters U and V . Label the newly formed cluster (UV) . Update the entries in the distance matrix by (a) deleting the rows and columns corresponding to clusters U and V , and (b) adding a row and column giving the distances between the cluster (UV) and the remaining clusters.
- 4 Repeat Steps 2 and 3 a total of $N - 1$ times. All objects will be in a single cluster after the algorithm terminates. Record the identity of clusters that are merged and the levels (distances or similarities) at which the mergers take place.

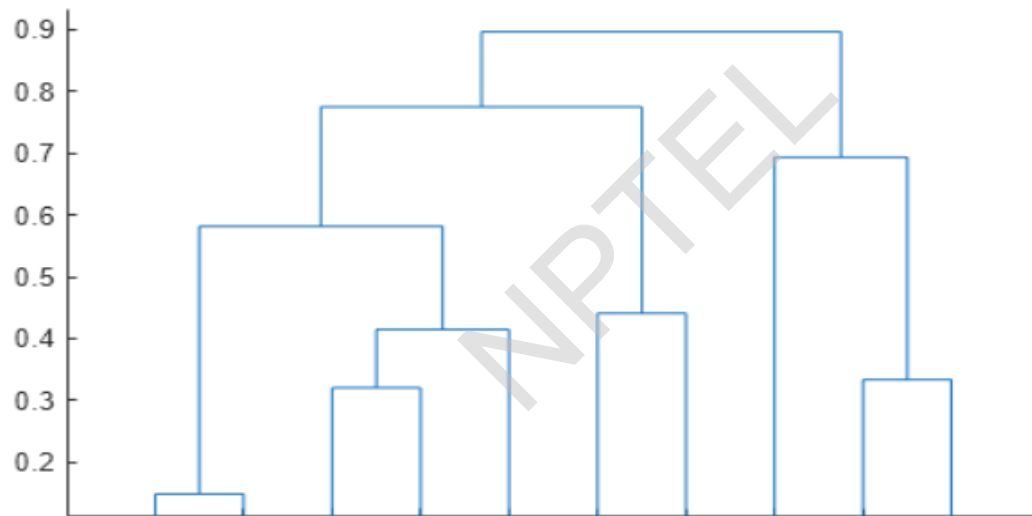
Example of Agglomerative Clustering

- Rain droplets combine to form small water patches, which eventually merge into larger puddles.
- Individuals at a social gathering initially stand alone, then form small discussion groups, which may merge into larger groups based on shared interests.
- Small settlements grow into villages, merge into towns, and eventually become large cities or metropolitan areas.

Dendrogram

- A dendrogram is a tree-like diagram used to visualize the sequence of merges in hierarchical clustering.
- Each observation starts as its own individual cluster at the bottom of the diagram.
- As the algorithm proceeds, clusters that are similar (based on linkage) are merged step-by-step.
- The height at which two clusters merge represents the dissimilarity (distance) between them.
- A taller merge height indicates less similarity between the two clusters being joined.
- The final single cluster at the top represents the point where all observations are joined.
- Cutting the dendrogram horizontally at a chosen height allows the extraction of k clusters.
- Dendrograms help identify natural cluster groupings and detect outliers or long chains.

Dendrogram (a general picture)



Algorithm

- 1 Begin with n observations and a measure (such as Euclidean distance) of all the $\binom{n}{2} = n(n-1)/2$ pairwise dissimilarities. Treat each observation as its own cluster.
- 2 For $i = n, n-1, \dots, 2$
 - 1 Examine all pairwise inter-cluster dissimilarities among the i clusters and identify the pair of clusters that are least dissimilar (that is, most similar). Fuse these two clusters. The dissimilarity between these two clusters indicates the height in the dendrogram at which the fusion should be placed.
 - 2 Compute the new pairwise inter-cluster dissimilarities among the $i-1$ remaining clusters.

Single Linkage

The inputs to a single linkage algorithm are distances or similarities between pairs of objects. Groups are formed from the individual entities by merging nearest neighbors, where the term *nearest neighbor* denotes the smallest distance or largest similarity.

Initially, we find the smallest distance in

$$D = \{d_{ik}\}$$

and merge the corresponding objects, say U and V , to obtain cluster (UV) .

For Step 3 of the general algorithm, the distances between (UV) and any other cluster W are computed as

$$d_{(UV)W} = \min\{d_{UW}, d_{VW}\}.$$

Here, d_{UW} and d_{VW} are the distances between the nearest neighbors of clusters U and W , and clusters V and W , respectively.

The results of single linkage clustering can be graphically displayed in the form of a dendrogram, or tree diagram. Branches in the tree represent clusters. The branches merge at nodes whose positions along a distance (or similarity) axis indicate the level at which the fusions occur.

Dendrograms for some examples are considered in the following sections.

Example

To illustrate the single linkage algorithm, consider the hypothetical distances between five objects. The distance matrix $D = \{d_{ik}\}$ is

$$D = \begin{bmatrix} 0 & & & & \\ 9 & 0 & & & \\ 3 & 7 & 0 & & \\ 6 & 5 & 9 & 0 & \\ 11 & 10 & 2 & 8 & 0 \end{bmatrix}$$

We treat each object as a separate cluster and merge the two closest objects. Since

$$\min_{i,k} (d_{ik}) = d_{53} = 2,$$

objects 5 and 3 are merged to form the cluster (35).

Example (continued)

To proceed, we need the distances between cluster (35) and the remaining clusters 1, 2, and 4. The nearest-neighbor distances are

$$d_{(35)1} = \min(d_{31}, d_{51}) = \min(3, 11) = 3,$$

$$d_{(35)2} = \min(d_{32}, d_{52}) = \min(7, 10) = 7,$$

$$d_{(35)4} = \min(d_{34}, d_{54}) = \min(9, 8) = 8.$$

After deleting the rows and columns for objects 3 and 5 and inserting the new cluster (35), the updated distance matrix becomes:

	(35)	1	2	4
(35)	0			
1	3	0		
2	7	9	0	
4	8	6	5	0

Example (continued)

The smallest distance is

$$d_{(35)1} = 3,$$

so clusters 1 and (35) merge to form (135).

Distances to other clusters:

$$d_{(135)2} = \min(d_{12}, d_{(35)2}) = \min(9, 7) = 7,$$

$$d_{(135)4} = \min(d_{14}, d_{(35)4}) = \min(6, 7) = 6.$$

The updated distance matrix is:

	(135)	2	4
(135)	0		
2	7	0	
4	6	5	0

Example (continued)

The smallest nearest-neighbor distance is

$$d_{42} = 5,$$

so clusters 4 and 2 merge to form cluster (24).

Now we have two clusters: (135) and (24). Their distance is

$$d_{(135)(24)} = \min(d_{(135)2}, d_{(135)4}) = \min(7, 6) = 6.$$

The final distance matrix is:

	(135)	(24)
(135)	0	
(24)	6	0

Example (continued)

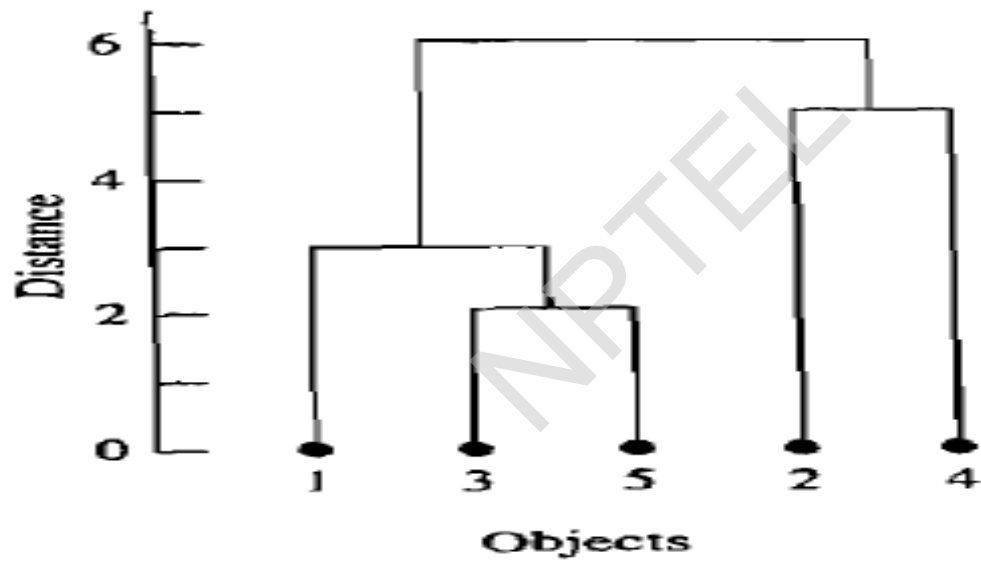
Finally, clusters (135) and (24) merge to form a single cluster of all five objects

(1 2 3 4 5),

when their nearest-neighbor distance reaches 6.

This illustrates the full sequence of merges for single linkage clustering.

Dendrogram



Complete Linkage

The general agglomerative algorithm starts by finding the minimum distance in $D = \{d_{ik}\}$ and merging the corresponding objects, such as U and V , to get cluster (UV) . For Step 3 of the general algorithm, the distances between (UV) and any other cluster W are computed by

$$d_{(UV)W} = \max\{d_{UW}, d_{VW}\}.$$

Here the quantities d_{UW} and d_{VW} are the distances between the most distant members of clusters U and W , and clusters V and W , respectively.

Example

Let us return to the distance matrix introduced in the previous example :

$$D = \begin{bmatrix} 0 & & & & \\ 9 & 0 & & & \\ 3 & 7 & 0 & & \\ 6 & 5 & 9 & 0 & \\ 11 & 10 & 2 & 8 & 0 \end{bmatrix}$$

At the first stage, objects 3 and 5 are merged, since they are most similar. This gives the cluster (35).

At stage 2, we compute

$$d_{(35)1} = \max(d_{31}, d_{51}) = \max(3, 11) = 11,$$

$$d_{(35)2} = \max(d_{32}, d_{52}) = 10,$$

$$d_{(35)4} = \max(d_{34}, d_{54}) = 9,$$

Example (continued)

The modified distance matrix becomes

	(35)	1	2	4
(35)	0			
1	11	0		
2	10	9	0	
4	9	6	5	0

The next merger occurs between the most similar groups, 2 and 4, to give the cluster (24).
At stage 3, we have

$$d_{(24)(35)} = \max(d_{2(35)}, d_{4(35)}) = \max(10, 9) = 10,$$
$$d_{(24)1} = \max(d_{21}, d_{41}) = 9,$$

Example (continued)

Now the distance matrix becomes

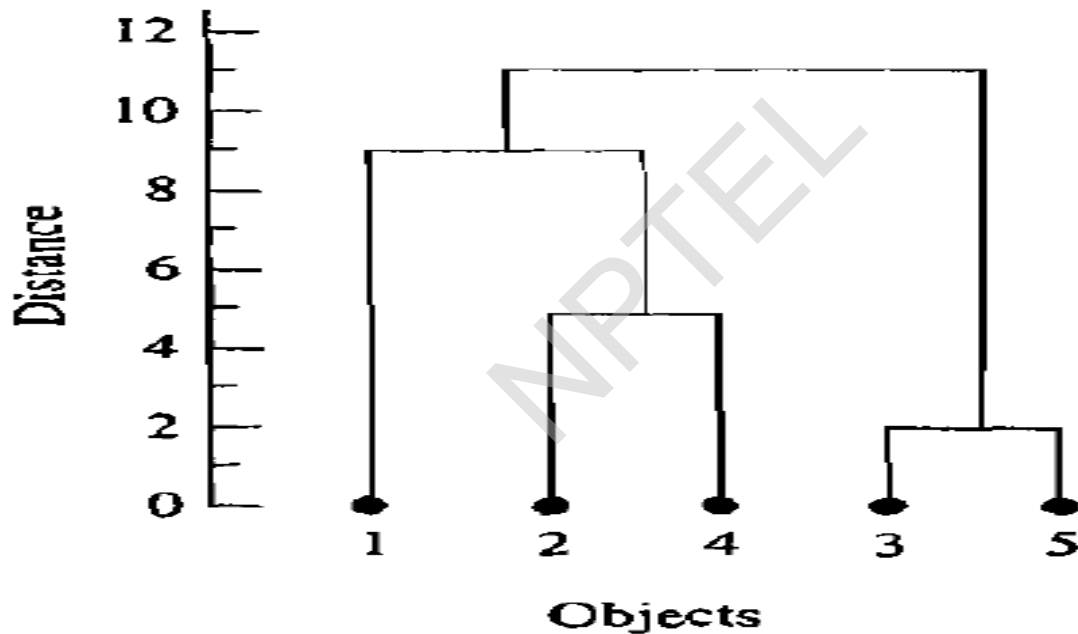
	(35)	(24)	1
(35)	0		
(24)	10	0	
1	11	9	0

The next merger produces the cluster (124).

At the final stage, the groups (35) and (124) are merged as the single cluster (12345) at level

$$d_{(124)(35)} = \max(d_{1(35)}, d_{(24)(35)}) = \max(11, 10) = 11.$$

Dendrogram



Average Linkage

Average linkage treats the distance between two clusters as the average distance between all pairs of items where one member of a pair belongs to each cluster.

Again, the input to the average linkage algorithm may be distances or similarities, and the method can be used to group objects or variables. The average linkage algorithm proceeds in the manner of the general algorithm. We begin by searching the distance matrix $D = \{d_{ik}\}$ to find the nearest (most similar) objects— for example, U and V . These objects are merged to form the cluster (UV) . For Step 3 of the general agglomerative algorithm, the distances between (UV) and the other cluster W are determined by

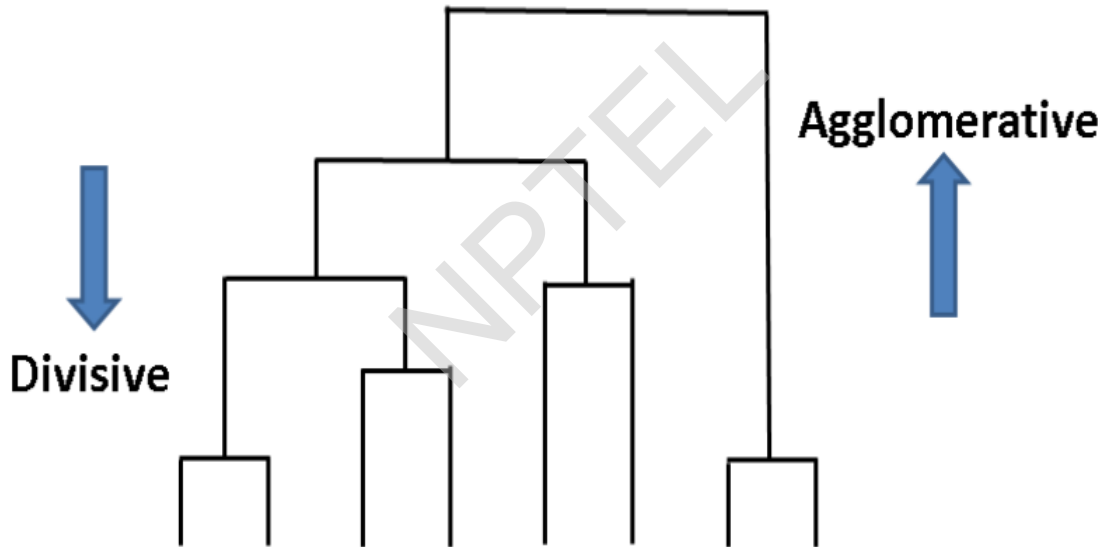
$$d_{(UV)W} = \frac{\sum_i \sum_k d_{ik}}{N_{(UV)} N_W}$$

where d_{ik} is the distance between object i in the cluster (UV) and object k in the cluster W , and $N_{(UV)}$ and N_W are the number of items in clusters (UV) and W , respectively.

Linkage

<i>Linkage</i>	<i>Description</i>
Complete	Maximal intercluster dissimilarity. Compute all pairwise dissimilarities between the observations in cluster A and the observations in cluster B, and record the <i>largest</i> of these dissimilarities.
Single	Minimal intercluster dissimilarity. Compute all pairwise dissimilarities between the observations in cluster A and the observations in cluster B, and record the <i>smallest</i> of these dissimilarities. Single linkage can result in extended, trailing clusters in which single observations are fused one-at-a-time.
Average	Mean intercluster dissimilarity. Compute all pairwise dissimilarities between the observations in cluster A and the observations in cluster B, and record the <i>average</i> of these dissimilarities.

Difference of Dendrogram



Divisive Hierarchical Clustering

- A top-down hierarchical clustering method.
- Starts with all objects in a single cluster.
- At each step, the cluster that is least homogeneous is selected for splitting.
- Splitting is usually based on maximizing inter-cluster distance or minimizing intra-cluster similarity.
- Continues dividing clusters until each object stands alone or a stopping criterion is reached.
- Produces a dendrogram that expands downward from a single root.
- Computationally more expensive than agglomerative clustering.
- Less commonly used in practice due to high complexity.

Example of Divisive Clustering

- A mixed basket of fruits is first separated from vegetables, then fruits are further split into categories such as sweet vs. sour, and later into subclasses like small vs. large, whereas vegetables are split into categories such as green vs. non-green.
- A big classroom of students is split based on subject preference and then further divided based on skill levels.
- A large rock is broken into smaller rocks, then into gravel, and eventually into fine sand.
- Animal taxonomy starts from a broad group (kingdom) and is successively divided into phylum, class, order, family, genus, and species.

References

- 1 Johnson, R. A., & Wichern, D. W. (2018). Applied multivariate statistical analysis (6th ed.). Pearson.
- 2 Han, J., Kamber, M., & Pei, J. (2012). Data mining: Concepts and techniques (3rd ed.). Morgan Kaufmann.
- 3 Everitt, B. S., Landau, S., Leese, M., & Stahl, D. (2011). Cluster analysis (5th ed.). Wiley.
- 4 Hastie, T., Tibshirani, R., & Friedman, J. (2009). The elements of statistical learning: Data mining, inference, and prediction (2nd ed.). Springer.

Thank you





SWAYAM NPTEL COURSE ON STATISTICAL FOUNDATION FOR BIG DATA ANALYSIS

BY
PROF. ARINDAM BANERJEE
Department of Mathematics
INDIAN INSTITUTE OF TECHNOLOGY KHARAGPUR

**Module 08: Multivariate Analysis And
Dimensionality**

Lecture 03: K-Means Clustering



Motivation of K -means clustering

K -means clustering groups similar data points into clusters without needing labeled data. It is used to uncover hidden patterns when the goal is to organize data based on similarity.

- It is a non-hierarchical clustering method.
- Helps identify natural groupings in unlabeled datasets.
- Works by grouping points based on distance to cluster centers.
- Commonly used in customer segmentation, image compression, and pattern discovery.
- Useful when you need structure from raw, unorganized data.

K-Means Clustering Algorithm

- Choose the number of clusters K .
- Select K initial centroids randomly from the dataset.
- Assign each data point to the nearest centroid.
- Update each centroid by taking the mean of all points assigned to that cluster.
- Repeat the assignment and update steps until the centroids do not change (convergence).
- Output the final clusters and their centroids.

K-means Clustering

- **Goal:** Partition n observations into k clusters so that points in the same cluster are similar.
- **Input:** Dataset $X = \{x_1, \dots, x_n\}$ and chosen number of clusters k .
- **Initialization:** Choose k initial centroids randomly.
- **Assignment step:** Assign each point to the nearest centroid (usually by Euclidean distance).
- **Update step:** Recompute each centroid as the mean of points assigned to it.

Clusters

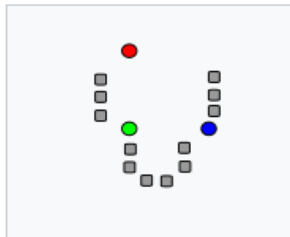
The K -means clustering procedure results from a simple and intuitive mathematical problem. We begin by defining some notation. Let $C_1, \dots, C_K$ denote sets containing the indices of the observations in each cluster. These sets satisfy two properties:

- $C_1 \cup C_2 \cup \dots \cup C_K = \{1, 2, \dots, n\}$. In other words, each observation belongs to at least one of the K clusters.
- $C_k \cap C_{k'} = \emptyset$ for all $k \neq k'$. In other words, the clusters are nonoverlapping: no observation belongs to more than one cluster.

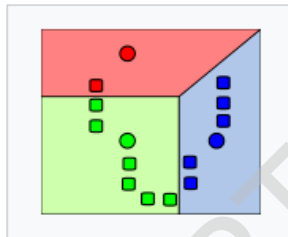
If the i th observation is in the k -th cluster, then $i \in C_k$. The idea behind K -means clustering is that a good clustering is one for which the within-cluster variation is as small as possible.

A plot of K - means clustering

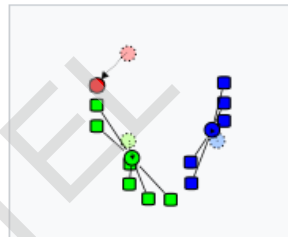
Demonstration of the standard algorithm



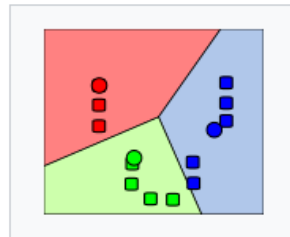
1. k initial "means" (in this case $k=3$) are randomly generated within the data domain (shown in color).



2. k clusters are created by associating every observation with the nearest mean. The partitions here represent the [Voronoi diagram](#) generated by the means.



3. The [centroid](#) of each of the k clusters becomes the new mean.



4. Steps 2 and 3 are repeated until convergence has been reached.

Courtesy: Wikipedia

Examples

We are given the dataset:

$$X = \{2, 4, 5, 10, 12, 13\}$$

We choose the number of clusters:

$$K = 2.$$

Step 1: Initialize Centroids

Choose two points at random as the initial centroids:

$$C_1^{(0)} = 4, \quad C_2^{(0)} = 12.$$

Step 2: Compute Distances and Assign Points

Compute distances using the Euclidean metric:

$$d(x, C_j) = |x - C_j|.$$

Point	$ x - 4 $	$ x - 12 $	Assigned
2	2	10	C_1
4	0	8	C_1
5	1	7	C_1
10	6	2	C_2
12	8	0	C_2
13	9	1	C_2

Thus,

$$C_1^{(1)} = \{2, 4, 5\}, \quad C_2^{(1)} = \{10, 12, 13\}.$$

Step 3: Update Centroids

Compute new centroids using the cluster means:

$$C_1^{(1)} = \frac{2 + 4 + 5}{3} = \frac{11}{3} = 3.67,$$

$$C_2^{(1)} = \frac{10 + 12 + 13}{3} = \frac{35}{3} = 11.67.$$

Step 4: Reassign Points Using Updated Centroids

Updated centroids:

$$C_1^{(1)} = 3.67, \quad C_2^{(1)} = 11.67.$$

Point	$ x - 3.67 $	$ x - 11.67 $	Assigned
2	1.67	9.67	C_1
4	0.33	7.67	C_1
5	1.33	6.67	C_1
10	6.33	1.67	C_2
12	8.33	0.33	C_2
13	9.33	1.33	C_2

No changes occur in the assignments, so the algorithm converges.

Final Result

The final clusters are:

$$C_1 = \{2, 4, 5\},$$

$$C_2 = \{10, 12, 13\}.$$

Final centroids:

$$C_1 = 3.67,$$

$$C_2 = 11.67.$$

Example

We are given the points:

$$\begin{aligned} A &= (1, 1), & B &= (1.5, 2), & C &= (3, 4), \\ D &= (5, 7), & E &= (3.5, 5), & F &= (4.5, 5). \end{aligned}$$

We choose:

$$K = 2.$$

Step 1: Initialize Centroids

We pick the initial centroids as:

$$\mu_1^{(0)} = A = (1, 1), \quad \mu_2^{(0)} = D = (5, 7).$$

Step 2: Distance Computation and Assignments

Distances use:

$$d^2(x, y) = (x_1 - y_1)^2 + (x_2 - y_2)^2.$$

Point	$d^2(x, \mu_1^{(0)})$	$d^2(x, \mu_2^{(0)})$	Assigned
A	0	52	C_1
B	1.25	37.25	C_1
C	13	13	C_2 (tie rule)
D	52	0	C_2
E	22.25	6.25	C_2
F	28.25	4.25	C_2

Thus:

$$C_1^{(0)} = \{A, B\}, \quad C_2^{(0)} = \{C, D, E, F\}.$$

Step 3: Update Centroids

Compute means:

$$\mu_1^{(1)} = \left(\frac{1 + 1.5}{2}, \frac{1 + 2}{2} \right) = (1.25, 1.5).$$

$$\mu_2^{(1)} = \left(\frac{3 + 5 + 3.5 + 4.5}{4}, \frac{4 + 7 + 5 + 5}{4} \right) = (4, 5.25).$$

Step 4: Reassign Points Using Updated Centroids

$$\mu_1^{(1)} = (1.25, 1.5), \quad \mu_2^{(1)} = (4, 5.25).$$

Point	$d^2(x, \mu_1^{(1)})$	$d^2(x, \mu_2^{(1)})$	Assigned
A	0.3125	27.0625	C_1
B	0.3125	16.8125	C_1
C	9.3125	2.5625	C_2
D	44.3125	4.0625	C_2
E	17.3125	0.3125	C_2
F	22.8125	0.3125	C_2

Assignments unchanged, so k-means converges.

Final Clusters

Final clusters:

$$C_1 = \{A(1, 1), B(1.5, 2)\},$$

$$C_2 = \{C(3, 4), D(5, 7), E(3.5, 5), F(4.5, 5)\}.$$

Final centroids:

$$\mu_1 = (1.25, 1.5), \quad \mu_2 = (4, 5.25).$$

Example

$$\begin{aligned} P_0 &= (1, 1), & P_1 &= (1.5, 2), & P_2 &= (2, 1), \\ P_3 &= (5, 5), & P_4 &= (6, 5), & P_5 &= (5, 6), \\ P_6 &= (8, 1), & P_7 &= (9, 2), & P_8 &= (8, 2), & P_9 &= (9, 1) \end{aligned}$$

Initial centroids:

$$c_0^{(0)} = (7.0318, 4.2734), \quad c_1^{(0)} = (6.4988, 5.0722), \quad c_2^{(0)} = (9.1081, -0.2512)$$

Iteration 0 — distances to initial centroids and assignment

For each point I list the three squared distances (to $c_0^{(0)}$, $c_1^{(0)}$, $c_2^{(0)}$) and the chosen (smallest) centroid.

idx	point	$d^2(\cdot, c_0^{(0)})$	$d^2(\cdot, c_1^{(0)})$	$d^2(\cdot, c_2^{(0)})$	assigned
0	(1.0, 1.0)	47.097531	46.820088	67.307050	c_1 (index 1)
1	(1.5, 2.0)	35.768908	34.426843	62.951431	c_1
2	(2.0, 1.0)	36.034000	36.822404	52.090831	c_0
3	(5.0, 5.0)	4.655976	2.251740	44.452137	c_1
4	(6.0, 5.0)	1.592445	0.254056	37.235918	c_1
5	(5.0, 6.0)	7.109118	3.107338	55.954627	c_1
6	(8.0, 1.0)	11.652815	18.836300	2.793520	c_2 (index 2)
7	(9.0, 2.0)	9.042427	15.694213	5.079792	c_2
8	(8.0, 2.0)	6.105958	11.691897	6.296011	c_0
9	(9.0, 1.0)	14.589285	22.838616	1.577302	c_2

Iteration 0 — Update centroids (compute cluster means)

Interpretation / assignment vector (iteration 0):

$$\text{assign}^{(0)} = [1, 1, 0, 1, 1, 1, 2, 2, 0, 2].$$

(That is: points 0,1,3,4,5 $\rightarrow c_1$; points 2,8 $\rightarrow c_0$; points 6,7,9 $\rightarrow c_2$.)

Iteration 0 $\rightarrow$ Update centroids (compute cluster means)

Take the mean of points assigned to each centroid.

Cluster for c_0 (indices 2,8):

$$c_0^{(1)} = \frac{(2.0, 1.0) + (8.0, 2.0)}{2} = \left(\frac{2+8}{2}, \frac{1+2}{2} \right) = (5.0, 1.5).$$

Cluster for c_1 (indices 0,1,3,4,5):

$$c_1^{(1)} = \frac{(1, 1) + (1.5, 2) + (5, 5) + (6, 5) + (5, 6)}{5} = \left(\frac{1 + 1.5 + 5 + 6 + 5}{5}, \frac{1 + 2 + 5 + 5 + 6}{5} \right) = \left(\frac{18.5}{5}, \frac{19}{5} \right) = (3.7, 3.8).$$

(Check: sum $x = 1 + 1.5 + 5 + 6 + 5 = 18.5$; sum $y = 1 + 2 + 5 + 5 + 6 = 19$.)

Cluster for c_2 (indices 6,7,9):

$$c_2^{(1)} = \frac{(8, 1) + (9, 2) + (9, 1)}{3} = \left(\frac{8 + 9 + 9}{3}, \frac{1 + 2 + 1}{3} \right) = \left(\frac{26}{3}, \frac{4}{3} \right) \approx (8.6666667, 1.3333333).$$

So updated centroids after iteration 0:

$$c_0^{(1)} = (5.0, 1.5), \quad c_1^{(1)} = (3.7, 3.8), \quad c_2^{(1)} \approx (8.6666667, 1.3333333).$$

Iteration 1 — distances to the updated centroids and new assignment

Compute squared distances to $c_0^{(1)}$, $c_1^{(1)}$, $c_2^{(1)}$ and pick the smallest.

idx	point	$d^2(\cdot, c_0^{(1)})$	$d^2(\cdot, c_1^{(1)})$	$d^2(\cdot, c_2^{(1)})$	assigned
0	(1.0, 1.0)	16.250000	15.130000	58.888889	c_1 (3.7, 3.8)
1	(1.5, 2.0)	12.500000	8.080000	51.805556	c_1
2	(2.0, 1.0)	9.250000	10.730000	44.555556	c_0 (5.0, 1.5)
3	(5.0, 5.0)	12.250000	3.130000	26.888889	c_1
4	(6.0, 5.0)	13.250000	6.729999	20.555556	c_1
5	(5.0, 6.0)	20.250000	6.530000	35.222222	c_1
6	(8.0, 1.0)	9.250000	26.330000	0.555556	c_2 (8.6667, 1.3333)
7	(9.0, 2.0)	16.250000	31.330000	0.555556	c_2
8	(8.0, 2.0)	9.250000	21.730000	0.888889	c_2
9	(9.0, 1.0)	16.250000	35.930000	0.222222	c_2

Iteration 1 — Update centroids

Assignment vector (iteration 1):

$$\text{assign}^{(1)} = [1, 1, 0, 1, 1, 1, 2, 2, 2, 2].$$

(Compared to iteration 0: point 8 moved from c_0 to c_2 ; point 2 remains c_0 ; cluster sizes are now 2, 5, 3 $\rightarrow$ 2,5,4 after the move.)

Iteration 1 $\rightarrow$ Update centroids

Compute means of newly assigned clusters.

Cluster for c_0 (indices 2 only):

Looking at $\text{assign}^{(1)}$, cluster c_0 has points $\{2\}$ only.

$$c_0^{(2)} = (2.0, 1.0).$$

Cluster for c_1 (indices 0,1,3,4,5):

$$c_1^{(2)} = \frac{(1, 1) + (1.5, 2) + (5, 5) + (6, 5) + (5, 6)}{5} = (3.7, 3.8).$$

(same as before; no change)

Cluster for c_2 (indices 6,7,8,9):

$$c_2^{(2)} = \frac{(8, 1) + (9, 2) + (8, 2) + (9, 1)}{4} = \left(\frac{8+9+8+9}{4}, \frac{1+2+2+1}{4} \right) = (8.5, 1.5).$$

So updated centroids after iteration 1:

$$c_0^{(2)} = (2.0, 1.0), \quad c_1^{(2)} = (3.7, 3.8), \quad c_2^{(2)} = (8.5, 1.5).$$

Iteration 2 — distances to $c^{(2)}$ and assignment

Squared distances to $c_0^{(2)} = (2, 1)$, $c_1^{(2)} = (3.7, 3.8)$, $c_2^{(2)} = (8.5, 1.5)$:

idx	point	$d^2(\cdot, c_0^{(2)})$	$d^2(\cdot, c_1^{(2)})$	$d^2(\cdot, c_2^{(2)})$	assigned
0	(1.0, 1.0)	1.000000	15.129999	56.500000	c_0 (2, 1)
1	(1.5, 2.0)	1.250000	8.080000	49.250000	c_0
2	(2.0, 1.0)	0.000000	10.730000	42.500000	c_0
3	(5.0, 5.0)	25.000000	3.130000	24.500000	c_1 (3.7, 3.8)
4	(6.0, 5.0)	34.000000	6.729999	18.500000	c_1
5	(5.0, 6.0)	41.000000	6.530000	32.500000	c_1
6	(8.0, 1.0)	36.000000	26.330000	0.500000	c_2 (8.5, 1.5)
7	(9.0, 2.0)	65.000000	31.330000	0.500000	c_2
8	(8.0, 2.0)	37.000000	21.730000	0.500000	c_2
9	(9.0, 1.0)	49.000000	35.930000	0.500000	c_2

Assignment vector (iteration 2):

$$\text{assign}^{(2)} = [0, 0, 0, 1, 1, 1, 2, 2, 2, 2].$$

This differs again from iteration 1: now points 0 and 1 moved to c_0 (which is (2, 1)).

Compute means with $\text{assign}^{(2)}$.

Cluster for c_0 (indices 0, 1, 2):

$$c_0^{(3)} = \frac{(1, 1) + (1.5, 2) + (2, 1)}{3} = \left(\frac{1 + 1.5 + 2}{3}, \frac{1 + 2 + 1}{3} \right) = (1.5, 1.\bar{3}) \approx (1.5, 1.333333).$$

Cluster for c_1 (indices 3, 4, 5):

$$c_1^{(3)} = \frac{(5, 5) + (6, 5) + (5, 6)}{3} = \left(\frac{5 + 6 + 5}{3}, \frac{5 + 5 + 6}{3} \right) = (5.\bar{3}, 5.\bar{3}) \approx (5.33333333, 5.33333333).$$

Cluster for c_2 (indices 6, 7, 8, 9):

$$c_2^{(3)} = (8.5, 1.5) \quad (\text{unchanged}).$$

So **updated centroids after iteration 2:**

$$c_0^{(3)} = (1.5, 1.333333), \quad c_1^{(3)} = (5.33333333, 5.33333333), \quad c_2^{(3)} = (8.5, 1.5).$$

Iteration 3 — distances to $c^{(3)}$ and assignment (convergence check)

Compute squared distances to the centroids $c^{(3)}$; here are the squared distances (rounded):

idx	point	$d^2(\cdot, c_0^{(3)})$	$d^2(\cdot, c_1^{(3)})$	$d^2(\cdot, c_2^{(3)})$	assigned
0	(1.0, 1.0)	0.361111	37.555556	56.500000	c_0
1	(1.5, 2.0)	0.444444	25.805556	49.250000	c_0
2	(2.0, 1.0)	0.361111	29.888889	42.500000	c_0
3	(5.0, 5.0)	25.694444	0.222222	24.500000	c_1
4	(6.0, 5.0)	33.694444	0.555556	18.500000	c_1
5	(5.0, 6.0)	34.027778	0.555556	32.500000	c_1
6	(8.0, 1.0)	42.361111	25.888889	0.500000	c_2
7	(9.0, 2.0)	56.694444	24.555556	0.500000	c_2
8	(8.0, 2.0)	42.694444	18.222222	0.500000	c_2
9	(9.0, 1.0)	56.361111	32.222222	0.500000	c_2

Final result (converged)

Final clusters (by index and points):

- **Cluster 0 (c_0):** indices $\{0, 1, 2\} \rightarrow$ points $(1.0, 1.0), (1.5, 2.0), (2.0, 1.0)$
centroid $c_0 = (1.5, 1.\overline{3})$.
- **Cluster 1 (c_1):** indices $\{3, 4, 5\} \rightarrow$ points $(5.0, 5.0), (6.0, 5.0), (5.0, 6.0)$
centroid $c_1 = (5.\overline{3}, 5.\overline{3})$.
- **Cluster 2 (c_2):** indices $\{6, 7, 8, 9\} \rightarrow$ points $(8.0, 1.0), (9.0, 2.0), (8.0, 2.0), (9.0, 1.0)$
centroid $c_2 = (8.5, 1.5)$.

Final centroids (numerical):

$$c_0 = (1.5, 1.333333\dots), \quad c_1 = (5.33333333\dots, 5.33333333\dots), \quad c_2 = (8.5, 1.5).$$

Expectation Maximization

K means follows the expectation maximization to solve the problem. Here the objective function is :

$$J = \sum_{i=1}^m \sum_{k=1}^K w_{ik} \|x^i - \mu_k\|^2$$

where $w_{ik} = 1$ for data point x_i if it belongs to cluster k ; otherwise, $w_{ik} = 0$. Also, μ_k is the centroid of x_i 's cluster. First minimize J w.r.t. w_{ik} and treat μ_k fixed. Then we minimize J w.r.t. μ_k and treat w_{ik} fixed.

Expectation Maximization (continued)

Now

$$\frac{\partial J}{\partial w_{ik}} = \sum_{i=1}^m \sum_{k=1}^K \|x^i - \mu_k\|^2$$

This implies $w_{ik} = \begin{cases} 1 & \text{if } k = \operatorname{argmin}_j \|x^i - \mu_j\|^2 \\ 0 & \text{otherwise} \end{cases}$

And

$$\begin{aligned} \frac{\partial J}{\partial \mu_k} &= 2 \sum_{i=1}^m w_{ik} (x^i - \mu_k) = 0 \\ \Rightarrow \mu_k &= \frac{\sum_{i=1}^m w_{ik} x^i}{\sum_{i=1}^m w_{ik}} \end{aligned}$$

That translates to recomputing the centroid of each cluster to reflect the new assignments.

Notes on K -means clustering

- Clustering methods such as k -means rely on distance calculations to measure similarity between data points. Therefore, it is advisable to standardize the data so that the features have mean zero and unit variance, especially when the dataset contains attributes measured on different scales (e.g., age vs. income).
- Since k -means updates centroids iteratively and uses random initialization, different runs may produce different clustering results. The algorithm can become trapped in a local minimum rather than reaching the global optimum. Hence, it is recommended to run the algorithm multiple times with different initial centroid seeds and select the clustering outcome that results in the smallest within-cluster sum of squared distances.
- If the cluster assignments do not change between iterations, then the within-cluster variance also remains unchanged. Mathematically, this is expressed as:

$$\frac{1}{m_k} \sum_{i=1}^{m_k} \|x^i - \mu_{c^k}\|^2$$

References

- 1 Géron, A. (2019). Hands-on machine learning with scikit-learn, Keras, and TensorFlow (2nd ed.). O'Reilly Media.
- 2 Han, J., Kamber, M., & Pei, J. (2012). Data mining: Concepts and techniques (3rd ed.). Morgan Kaufmann.
- 3 Hastie, T., Tibshirani, R., & Friedman, J. (2009). The elements of statistical learning: Data mining, inference, and prediction (2nd ed.). Springer.

Thank you





SWAYAM NPTEL COURSE ON STATISTICAL FOUNDATION FOR BIG DATA ANALYSIS

BY
PROF. ARINDAM BANERJEE
Department of Mathematics
INDIAN INSTITUTE OF TECHNOLOGY KHARAGPUR

**Module 08: Multivariate Analysis And
Dimensionality**
Lecture 04: Curse of Dimensionality



Curse of Dimensionality

- "Curse of dimensionality" is a term that was first used by Richard E. Bellman back in the 1960s. It began as Bellman's idea from dynamic optimization and it turned out to be a fundamental concept for understanding complexity in high-dimensional spaces.
- It is at its core the difficulties and unique characteristics one faces when working with data in high-dimensional spaces.

Curse of Dimensionality (continued)

- When dimensionality increases, data points that were close together in lower dimensions become increasingly separated.
- This is due to the fact that the amount of space expands exponentially with each new addition of another dimension
- Just imagine a cube becoming a hypercube; its corners move further away from its center and make it more empty inside. This growing emptiness is what is referred to as **data sparsity**.

Curse of Dimensionality (continued)

- Metrics like Euclidean distances which are useful for measuring proximity between data points in lower dimensions, lose their effectiveness.
- This means that most pairs of points become nearly equidistant from each other and from a reference point.
- This convergence makes it harder to distinguish between close neighbors and distant ones.
- In tasks like classification, where distance measurements are crucial for categorizing new data points, these metrics become less effective.

Practical Implications

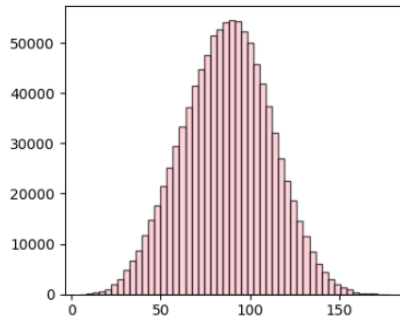
- One way the curse of dimensionality problems show up is overfitting.
- Overfitting happens when a complex model learns noise instead of the patterns in the data.
- The model can make false connections or correlations and perform poorly when it sees new data.
- The curse also makes it hard to find patterns in big datasets.
- High-dimensional data is spread out and sparse, so it's challenging for traditional analysis methods to find meaningful insights.

Solution

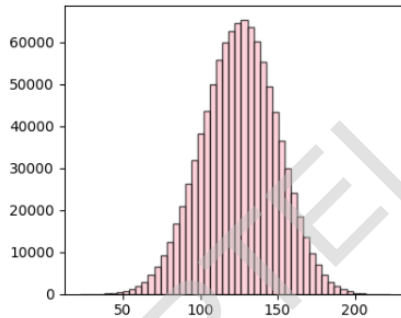
Dimension Reduction

Pairwise Distance in Different Dimensions

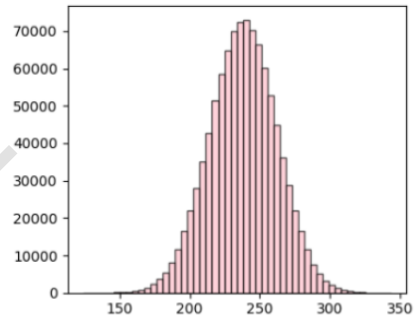
Pairwise Distances in 5D



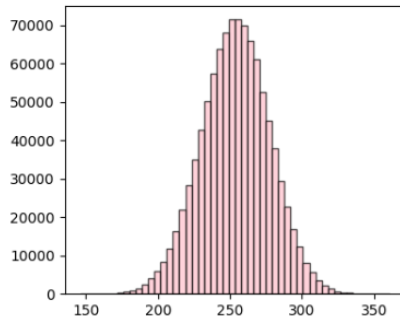
Pairwise Distances in 10D



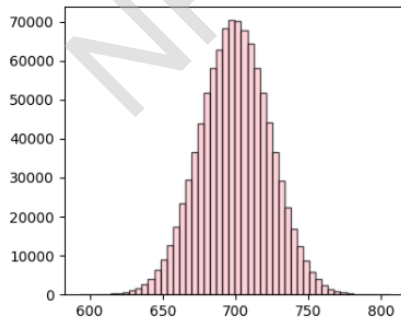
Pairwise Distances in 35D



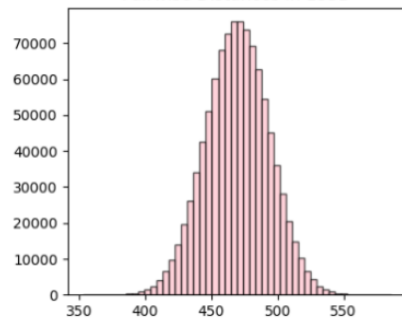
Pairwise Distances in 40D



Pairwise Distances in 300D

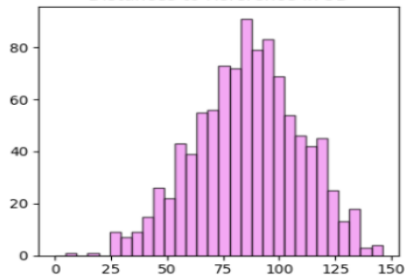


Pairwise Distances in 135D

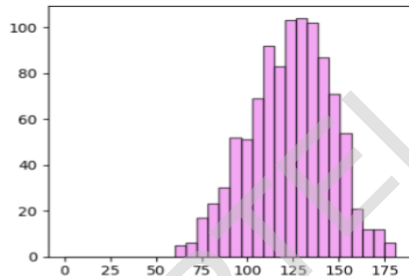


Distances to References in Different Dimensions

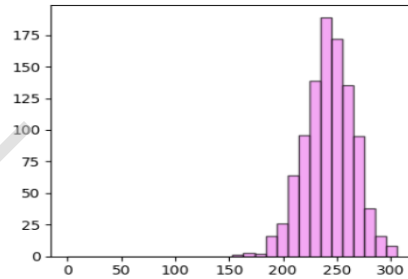
Distances to Reference in 5D



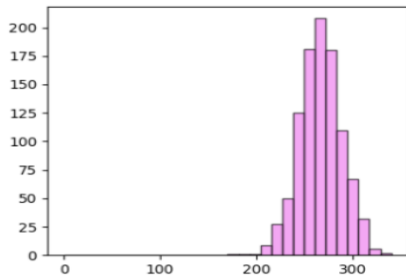
Distances to Reference in 10D



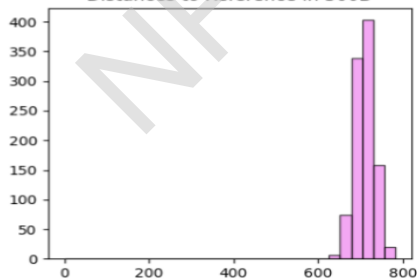
Distances to Reference in 35D



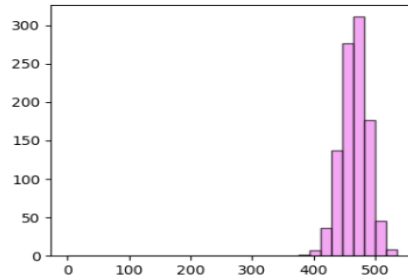
Distances to Reference in 40D



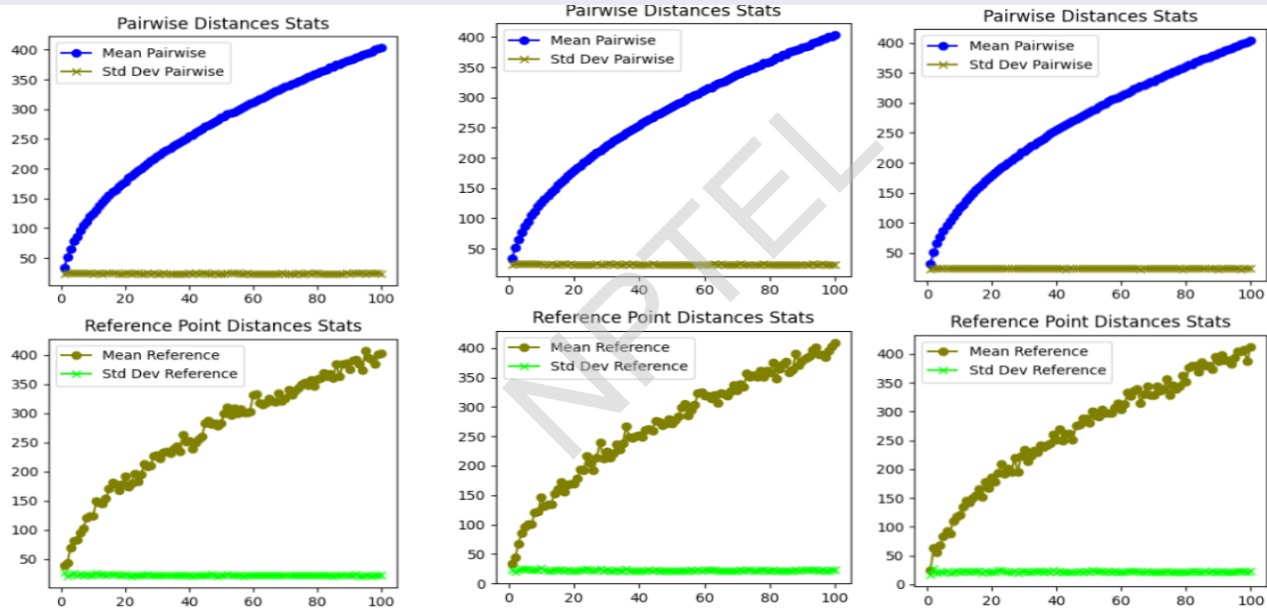
Distances to Reference in 300D



Distances to Reference in 135D



Pairwise Distance and Reference Point Distance Stats



References

- 1 Gorban, A. N., & Zinovyev, A. Y. (2008). Principal manifolds for data visualization and dimension reduction. Lecture Notes in Computational Science and Engineering, Springer.
- 2 Hastie, T., Tibshirani, R., & Friedman, J. (2009). The elements of statistical learning: Data mining, inference, and prediction (2nd ed.). Springer.
- 3 Verleysen, M., & François, D. (2014). High-dimensional data analysis: The curses and blessings (1st ed.). Wiley.

Thank you





SWAYAM NPTEL COURSE ON STATISTICAL FOUNDATION FOR BIG DATA ANALYSIS

BY
PROF. ARINDAM BANERJEE
Department of Mathematics
INDIAN INSTITUTE OF TECHNOLOGY KHARAGPUR

**Module 08: Multivariate Analysis And
Dimensionality**

**Lecture 05: Behaviour of Volume in High
Dimension**



Theorem

Consider the n -dimensional hypercube $[0, 1]^n$, of unit length. Let the volume of this cube be V_1 ($V_1 = 1$). Fix $r \in [0, 1)$ and take the smaller hypercube $[0, r]^n$, of length, r and let V_2 be its volume. Then the ratio $\frac{V_1 - V_2}{V_1} \rightarrow 1$ as $n \rightarrow \infty$.

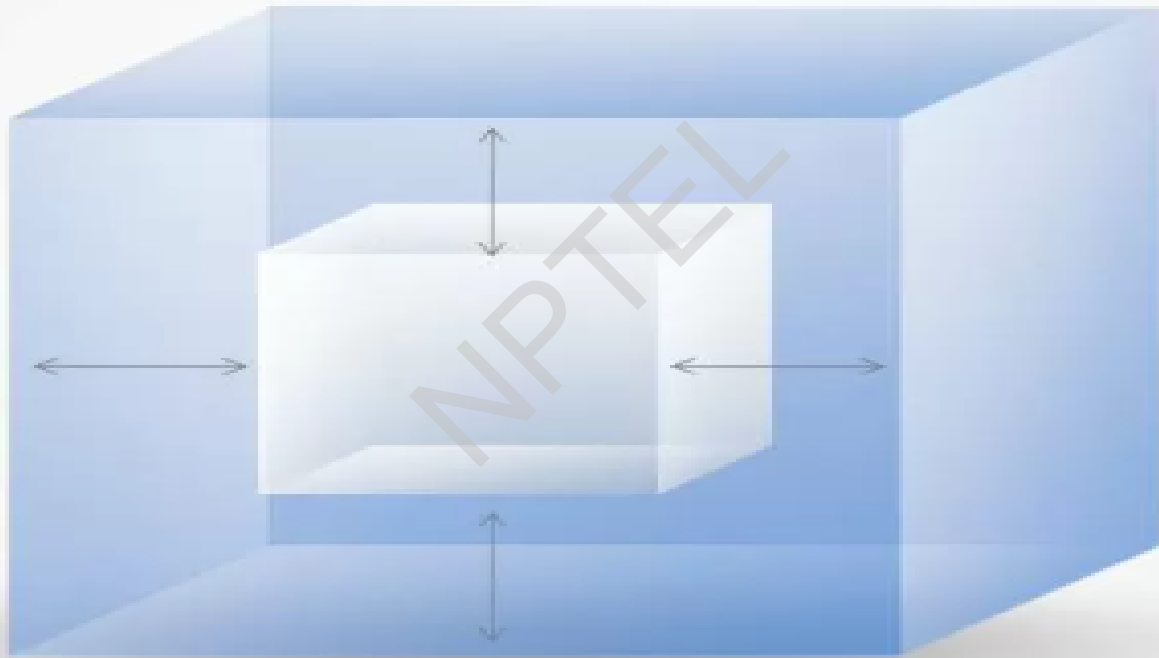
Proof

Here $V_1=1$ and $V_2 = r^n$. Now

$$\begin{aligned} & \lim_{n \rightarrow \infty} \frac{V_1 - V_2}{V_1} \\ &= \lim_{n \rightarrow \infty} \frac{1 - r^n}{1} \\ &= 1 - \lim_{n \rightarrow \infty} r^n \\ &= 1 \end{aligned}$$

In other words, asymptotically, the volume of the bigger hypercube is **concentrated** in the “remaining portion”.

Behaviour of Volume of Multidimensional Hypercube



Gamma Function

The gamma function is defined as

$$\Gamma(z) = \int_0^{\infty} t^{z-1} e^{-t} dt, \quad z > 0$$

Some of the important properties of gamma function are

- $\Gamma(z+1) = z \Gamma(z)$ (Functional equation)
- $\Gamma(n) = (n-1)!$ for $n \in \mathbb{N}$ (Factorial extension)
- $\Gamma(z)\Gamma(1-z) = \frac{\pi}{\sin(\pi z)}$ (Reflection formula)
- $\Gamma(z)\Gamma(z + \frac{1}{2}) = 2^{1-2z} \sqrt{\pi} \Gamma(2z)$ (Duplication formula)

Values of the Gamma Function

z	$\Gamma(z)$
1	1
2	$1! = 1$
3	$2! = 2$
4	$3! = 6$
$\frac{1}{2}$	$\sqrt{\pi}$
$\frac{3}{2}$	$\frac{1}{2}\sqrt{\pi}$
$\frac{5}{2}$	$\frac{3}{4}\sqrt{\pi}$

Theorem

Consider the n -dimensional hypersphere of unit radius and let its volume be V_1 ($V_1 = 1$). Fix $r \in [0, 1)$ and consider the smaller hypersphere of radius r . If the volume of the latter one is V_2 then the ratio $\frac{V_1 - V_2}{V_1} \rightarrow 1$ as $n \rightarrow \infty$.

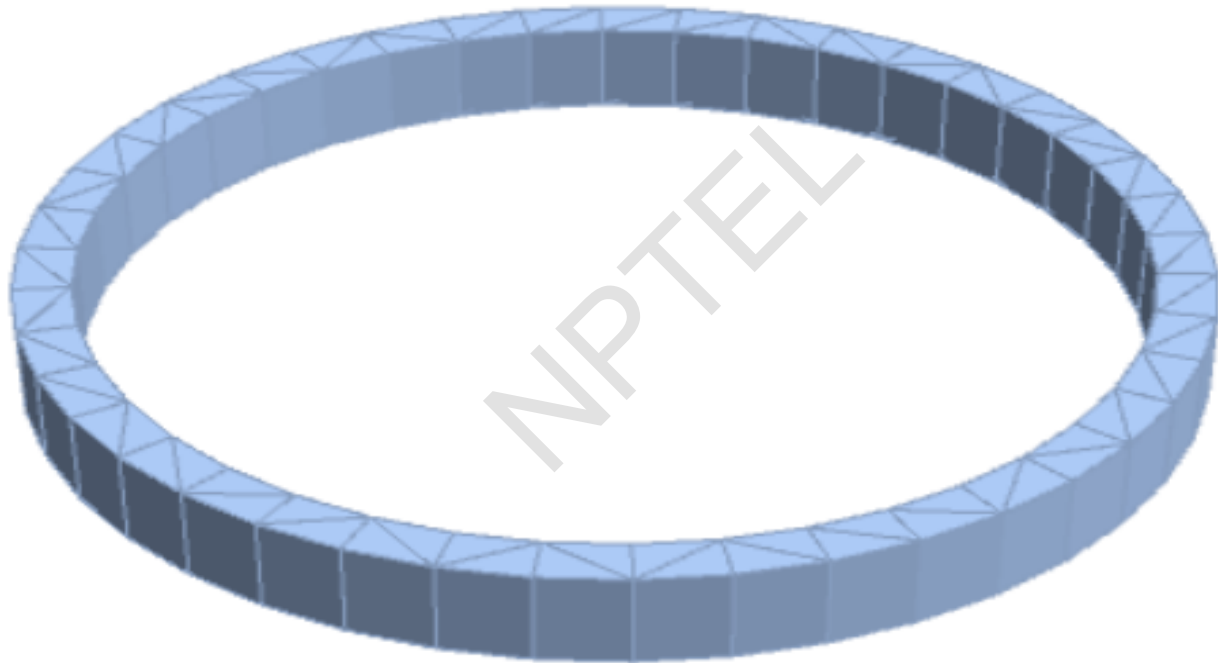
Proof

Here $V_1 = \frac{\pi^{n/2}}{\Gamma(n/2+1)}$ and $V_2 = \frac{\pi^{n/2}}{\Gamma(n/2+1)} r^n$. Where
Now,

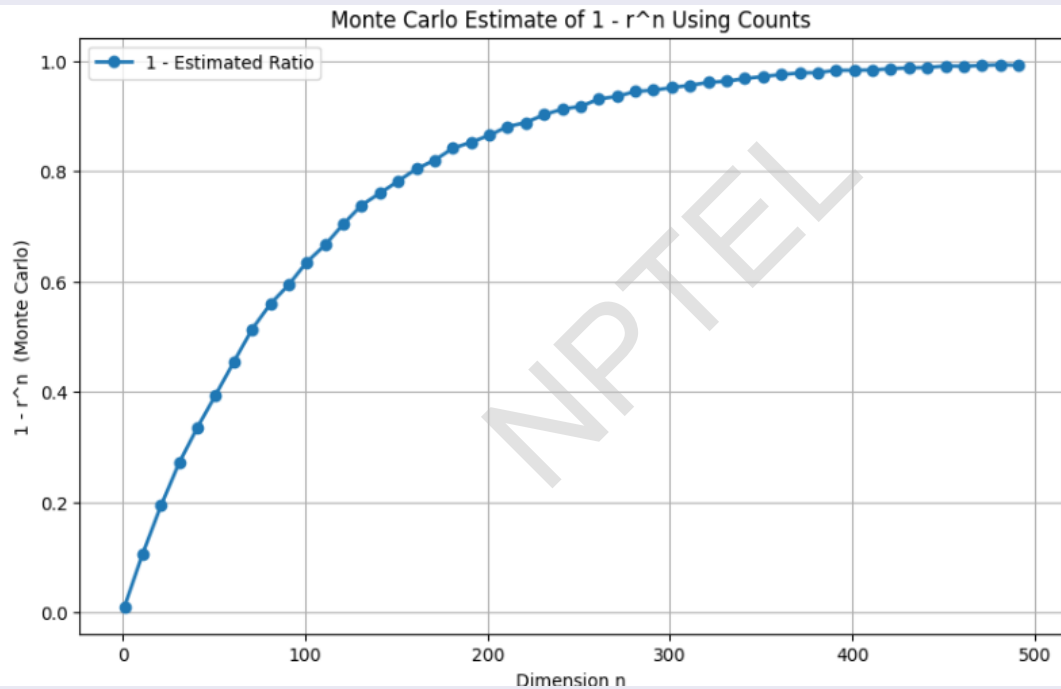
$$\begin{aligned} & \lim_{n \rightarrow \infty} \frac{V_1 - V_2}{V_1} \\ &= \lim_{n \rightarrow \infty} \frac{1 - r^n}{1} \quad \left[\text{the factor } \frac{\pi^{n/2}}{\Gamma(n/2+1)} \text{ will be cancelled out from numerator and denominator} \right] \\ &= 1 - \lim_{n \rightarrow \infty} r^n \\ &= 1 \end{aligned}$$

In other words, asymptotically, the volume of the bigger hypersphere is **concentrated** in the annular region.

Behaviour of Volume of Multidimensional Sphere



Behaviour of Volume of Multidimensional Hypercube



Table

	Dimension n	Inside Small Box	Outside Small Box	Estimated Ratio r^n
0	1	19792	208	0.98960
1	11	17869	2131	0.89345
2	21	16131	3869	0.80655
3	31	14573	5427	0.72865
4	41	13290	6710	0.66450
5	51	12112	7888	0.60560
6	61	10909	9091	0.54545
7	71	9727	10273	0.48635
8	81	8815	11185	0.44075
9	91	8122	11878	0.40610
10	101	7292	12708	0.36460
11	111	6675	13325	0.33375
12	121	5916	14084	0.29580
13	131	5232	14768	0.26160
14	141	4799	15201	0.23995
15	151	4362	15638	0.21810
16	161	3917	16083	0.19585
17	171	3604	16396	0.18020
18	181	3171	16829	0.15855
19	191	2952	17048	0.14760
20	201	2701	17299	0.13505
21	211	2381	17619	0.11905
22	221	2239	17761	0.11195
23	231	1955	18045	0.09775
24	241	1751	18249	0.08755
25	251	1651	18349	0.08255
26	261	1386	18614	0.06930
27	271	1294	18706	0.06470
28	281	1112	18888	0.05560
29	291	1065	18935	0.05325
30	301	970	19030	0.04850
31	311	886	19114	0.04430
32	321	782	19218	0.03910
33	331	738	19262	0.03690
34	341	646	19354	0.03230
35	351	569	19431	0.02845
36	361	501	19499	0.02505
37	371	439	19561	0.02195
38	381	430	19570	0.02150
39	391	343	19657	0.01715
40	401	342	19658	0.01710
41	411	331	19669	0.01655
42	421	286	19714	0.01430
43	431	256	19744	0.01280
44	441	245	19755	0.01225
45	451	184	19816	0.00920
46	461	194	19806	0.00970
47	471	166	19834	0.00830
48	481	143	19857	0.00715
49	491	164	19836	0.00820

High Dimensional Volume

- **Vast Volume:** In high dimensions, the available space grows exponentially compared to 2D or 3D.
- **Sparsity:** Data points become extremely spread out, making most of the high-dimensional volume empty space.
- **Curse of Dimensionality:** This sparsity and vastness challenges many standard machine learning algorithms.

K-Means Clustering: Challenges

- **Meaningless Distances:** Standard metrics like Euclidean distance become less effective; distances tend to converge.
- **Difficulty Forming Clusters:** Sparsity makes it hard to find naturally dense regions.
- **Noise Impact:** Irrelevant features can easily distort distance calculations and results.
- **Computational Cost:** The number of calculations increases significantly with more dimensions.

K-Means Clustering: Way Out

- **Dimensionality Reduction:** Principal Component Analysis.
- **Feature Selection:** Select only the most relevant features to mitigate the impact of noise.

Thank you

